



**Brit College of Engineering & Technology  
(BCET)**

## **Data Analytics Logbook**

**Research Project-5**

**Data Science and AI**

**(upervised ML – Classification 3)**

**Submitted by: Md.Akib Sultan**

**Student ID:**

**Email: akibsultan1045@gmail.com**

<https://colab.research.google.com/drive/14EQVlos8fbCHoavXAA9lCjp87-DnMuZW?usp=sharing>

# Supervised Classification for Heart Disease Prediction: A Comprehensive Tutorial

## Introduction to Supervised Classification

Supervised classification is a type of machine learning task where algorithms learn to map input data to an output label based on a labelled dataset. The "supervised" aspect comes from the fact that the algorithm is "guided" by having access to the correct answers (labels) during its training phase. Once trained, these models can predict the label for new, unseen data points.

In the medical domain, supervised classification is incredibly valuable for tasks like disease diagnosis, risk prediction, and treatment recommendation. For instance, predicting heart disease based on a patient's clinical parameters is a classic application, offering the potential for early intervention and improved patient outcomes.

## The Heart Disease Dataset

For this tutorial, we will use the popular Heart Disease dataset, often sourced from the UCI Machine Learning Repository. This dataset contains medical records of patients concerning heart disease presence.

**Dataset Source:** The dataset can be found on Kaggle, originally derived from the UCI Machine Learning Repository. **Link:** [Heart Disease UCI Dataset on Kaggle](#)

### Key Features (Columns) in the Dataset:

- age: age in years
- sex: (1 = male; 0 = female)
- cp: chest pain type (1: typical angina, 2: atypical angina, 3: non-anginal pain, 4: asymptomatic)
- trestbps: resting blood pressure (in mm Hg on admission to the hospital)
- chol: serum cholestoral in mg/dl
- fbs: fasting blood sugar > 120 mg/dl (1 = true; 0 = false)
- restecg: resting electrocardiographic results (0: normal, 1: ST-T wave abnormality, 2: definite left ventricular hypertrophy)
- thalach: maximum heart rate achieved
- exang: exercise induced angina (1 = yes; 0 = no)
- oldpeak: ST depression induced by exercise relative to rest

- slope: the slope of the peak exercise ST segment (1: upsloping, 2: flat, 3: downsloping)
- ca: number of major vessels (0-3) colored by flourosopy
- thal: 3 = normal; 6 = fixed defect; 7 = reversable defect
- target: diagnosis of heart disease (0 = no disease, 1 = disease) - This is our target variable.

### Google Colab Code: Step-by-Step Tutorial

# --- Step 0: Setup and Import Libraries ---

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler, OneHotEncoder
```

```
from sklearn.compose import ColumnTransformer
```

```
from sklearn.pipeline import Pipeline
```

```
from sklearn.impute import SimpleImputer # For handling '?' if present
```

# Classifiers

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.svm import SVC
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
```

# Metrics

```
from sklearn.metrics import (
```

```
    accuracy_score,
```

```

confusion_matrix,
roc_curve,
roc_auc_score,
classification_report
)

# Set display options for pandas
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000)

# Set a random state for reproducibility
RANDOM_STATE = 42
print("Libraries loaded successfully!")

# For simplicity, if not using opendatasets, upload 'heart_disease_uci.csv' directly to
Colab

print("Trying to load directly from current directory if uploaded manually..")
df = None # Initialize df to None for safety
try:
    df = pd.read_csv("/content/heart_disease_uci.csv")
    print("Dataset loaded from current directory.")
except FileNotFoundError:
    print("Please upload 'heart_disease_uci.csv' to your Google Colab environment.")
    print("Data loading failed. Skipping subsequent data display.")

    # Removed exit() to allow the notebook to continue execution and prompt user to
upload.

# Only proceed with displaying data if df was successfully loaded
if df is not None:
    print("Original Data Head:")

```

```
print(df.head())
```

```
print("\nOriginal Data Info:")
```

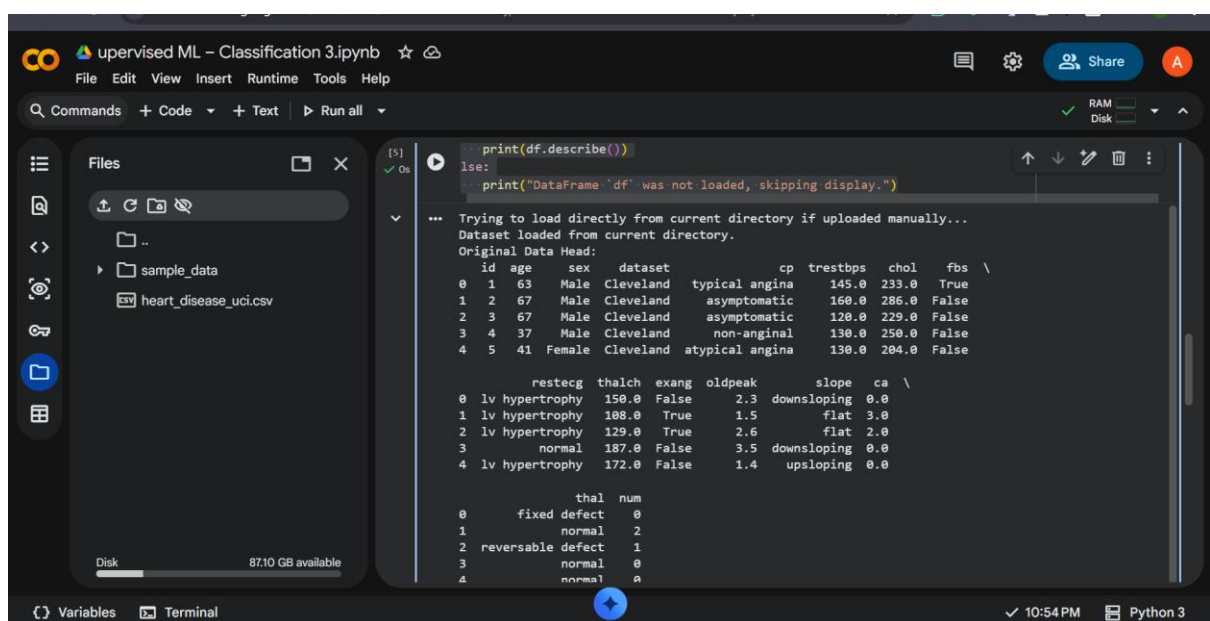
```
df.info()
```

```
print("\nOriginal Data Description:")
```

```
print(df.describe())
```

else:

```
print("DataFrame `df` was not loaded, skipping display.")
```



# --- Step 2: Data Cleaning, Preprocessing, and Wrangling ---

```
print("\n--- Step 2: Data Cleaning, Preprocessing, and Wrangling ---")
```

# Drop 'id' column as it's not useful for prediction

```
if 'id' in df.columns:
```

```
    df = df.drop('id', axis=1)
```

```
    print("Dropped 'id' column.")
```

# Handle missing values: The 'ca' and 'thal' columns in the UCI version often have '?'

# We'll convert '?' to NaN and then impute

```
df.replace('?', np.nan, inplace=True)

print("\nReplaced '?' with NaN.")
```

# Convert 'ca' and 'thal' to numeric, as they might have been read as object due to '?'

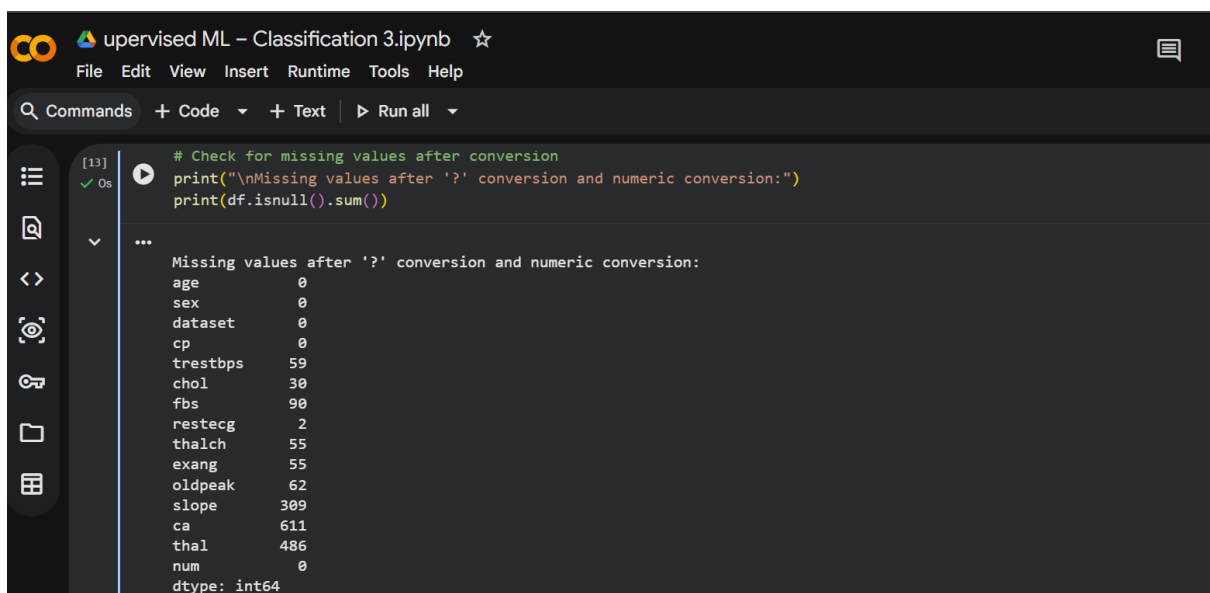
```
df['ca'] = pd.to_numeric(df['ca'])
df['thal'] = pd.to_numeric(df['thal'])

print("Converted 'ca' and 'thal' to numeric.")
```

# Check for missing values after conversion

```
print("\nMissing values after '?' conversion and numeric conversion:")

print(df.isnull().sum())
```



The screenshot shows a Jupyter Notebook window titled "upervised ML - Classification 3.ipynb". The code cell [13] is executed, showing the output of the `df.isnull().sum()` command. The output is a table of missing values for various features.

| Feature  | Missing Values |
|----------|----------------|
| age      | 0              |
| sex      | 0              |
| dataset  | 0              |
| cp       | 0              |
| trestbps | 59             |
| chol     | 30             |
| fbs      | 90             |
| restecg  | 2              |
| thalch   | 55             |
| exang    | 55             |
| oldpeak  | 62             |
| slope    | 309            |
| ca       | 611            |
| thal     | 486            |
| num      | 0              |

The output also includes the text "Missing values after '?' conversion and numeric conversion:" and "dtype: int64".

# Define target variable and features

```
X = df.drop('target', axis=1)

y = df['target']
```

# Ensure target is binary (some datasets have 0-4 scale, ours is 0/1)

# If target was 0-4, binarize it: `y = (y > 0).astype(int)`

```
print(f"\nTarget variable distribution before split:\n{y.value_counts()}")
```

```
[16]
✓ Os
# Ensure target is binary (some datasets have 0-4 scale, ours is 0/1)
# If target was 0-4, binarize it: y = (y > 0).astype(int)
print(f"\nTarget variable distribution before split:\n{y.value_counts()}")

...
Target variable distribution before split:
num
0    411
1    265
2    109
3    107
4     28
Name: count, dtype: int64
```

# Split data into training and testing sets

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=RANDOM_STATE, stratify=y)

print(f"\nTraining set shape: {X_train.shape}")
print(f"Testing set shape: {X_test.shape}")
print(f"Target distribution in training set:\n{y_train.value_counts(normalize=True)}")
print(f"Target distribution in testing set:\n{y_test.value_counts(normalize=True)}")
```

```
# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=RANDOM_STATE, stratify=y)
print(f"\nTraining set shape: {X_train.shape}")
print(f"Testing set shape: {X_test.shape}")
print(f"Target distribution in training set:\n{y_train.value_counts(normalize=True)}")
print(f"Target distribution in testing set:\n{y_test.value_counts(normalize=True)}")

...
Training set shape: (736, 14)
Testing set shape: (184, 14)
Target distribution in training set:
num
0    0.447011
1    0.288043
2    0.118207
3    0.116848
4    0.029891
Name: proportion, dtype: float64
Target distribution in testing set:
num
```

# Define numerical and categorical features

```
numerical_features = X.select_dtypes(include=np.number).columns.tolist()
```

```
categorical_features = X.select_dtypes(include='object').columns.tolist()
```

```
from sklearn.pipeline import Pipeline
```

```
from sklearn.impute import SimpleImputer
```

```
from sklearn.preprocessing import StandardScaler
```

```
# Define preprocessing steps using ColumnTransformer and Pipelines
```

```
# Numerical Pipeline: Impute with median (for robustness to outliers), then scale
```

```
numerical_transformer = Pipeline(steps=[  
    ('imputer', SimpleImputer(strategy='median')),  
    ('scaler', StandardScaler())  
])
```

```
# Categorical Pipeline: Impute with most frequent, then One-Hot Encode
```

```
categorical_transformer = Pipeline(steps=[  
    ('imputer', SimpleImputer(strategy='most_frequent')),  
    ('onehot', OneHotEncoder(handle_unknown='ignore')) # handle_unknown='ignore' for  
    new categories in test set  
])
```

```
#te a preprocessor using ColumnTransformer
```

```
preprocessor = ColumnTransformer(  
    transformers=[  
    ('num', numerical_transformer, numerical_features),  
    ('cat', categorical_transformer, categorical_features)  
])
```



```
print("\nDefined preprocessing pipelines for numerical and categorical features.")
```

```
# --- Step 3: Exploratory Data Analysis (EDA) ---
```

```
print("\n--- Step 3: Exploratory Data Analysis (EDA) ---"
```

```
# Target variable distribution
```

```
plt.figure(figsize=(6, 4))
```

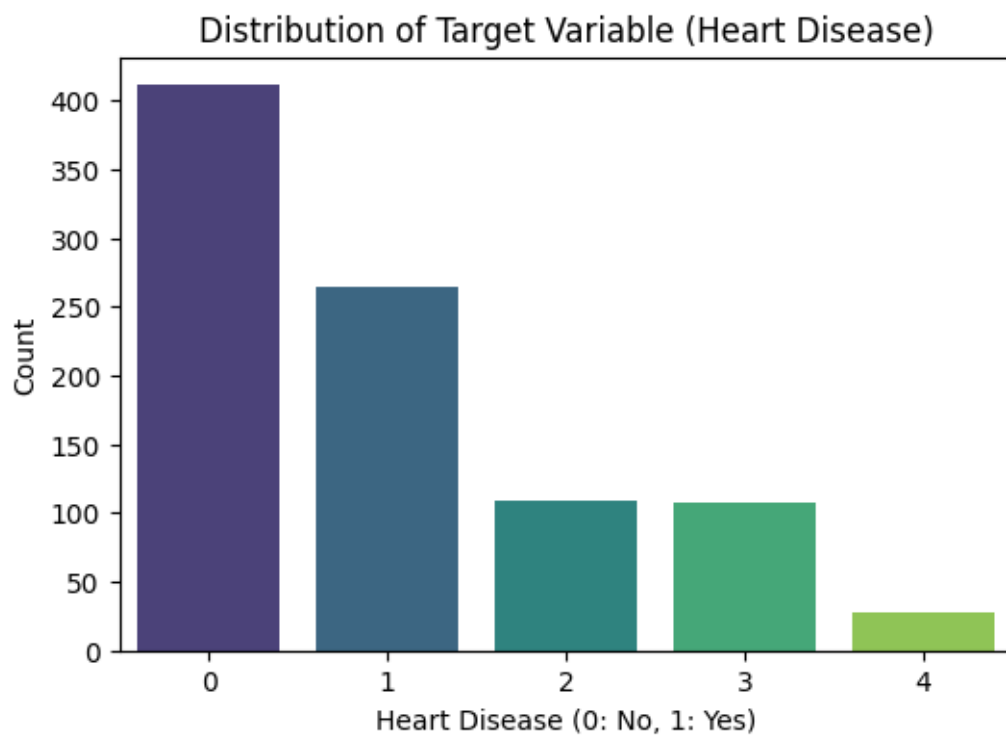
```
sns.countplot(x=y, palette='viridis')
```

```
plt.title('Distribution of Target Variable (Heart Disease)')
```

```
plt.xlabel('Heart Disease (0: No, 1: Yes)')
```

```
plt.ylabel('Count')
```

```
plt.show()
```



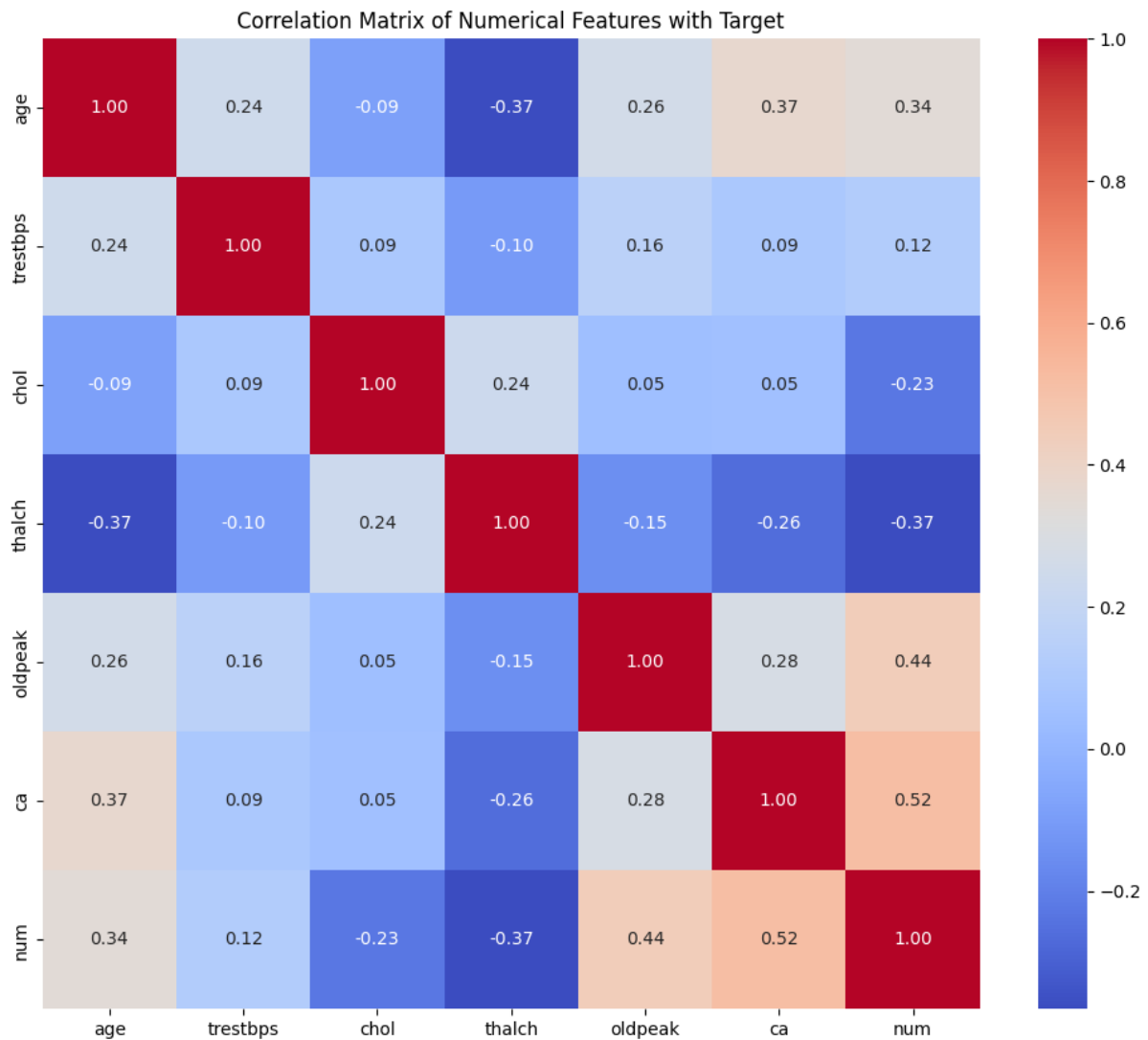
```
# Correlation Heatmap for numerical features
```

```
plt.figure(figsize=(12, 10))
```

```
sns.heatmap(df[numerical_features + ['target']].corr(), annot=True, cmap='coolwarm',  
fmt=".2f")
```

```
plt.title('Correlation Matrix of Numerical Features with Target')
```

```
plt.show()
```



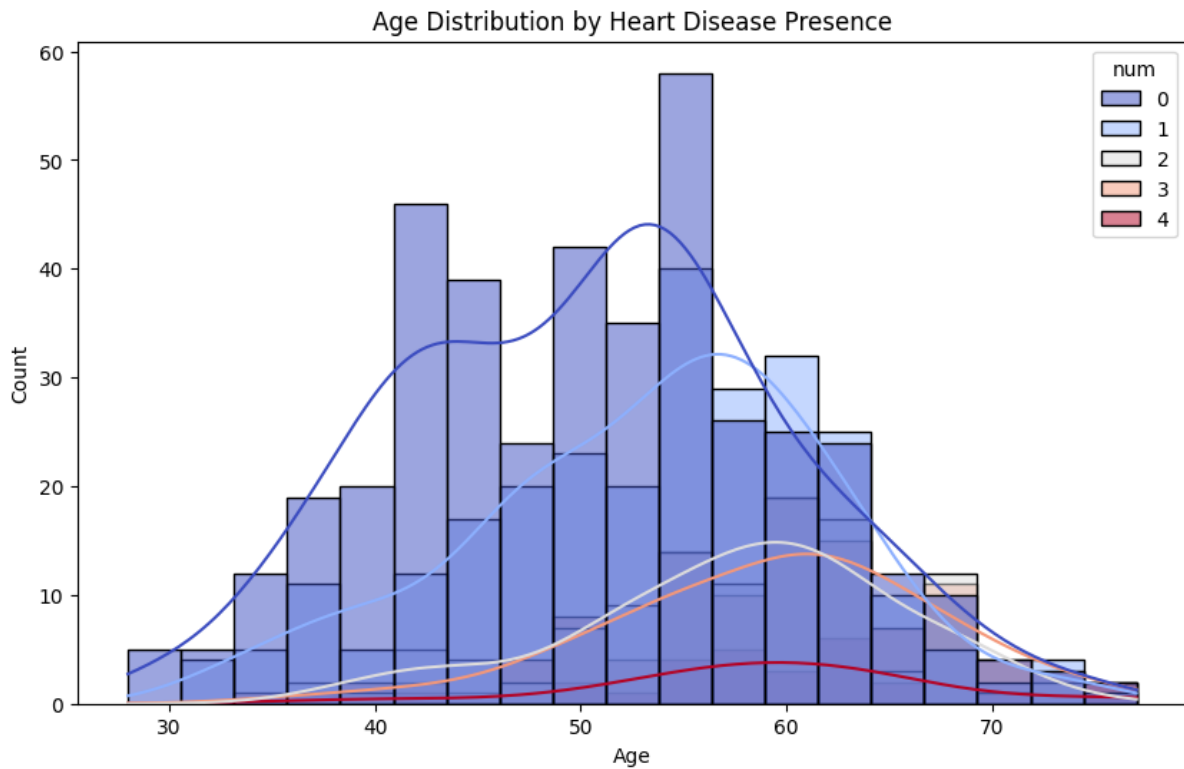
```
# Example: Distribution of 'age' by 'target'
```

```
plt.figure(figsize=(10, 6))
```

```
sns.histplot(data=df, x='age', hue='target', kde=True, palette='coolwarm')
```

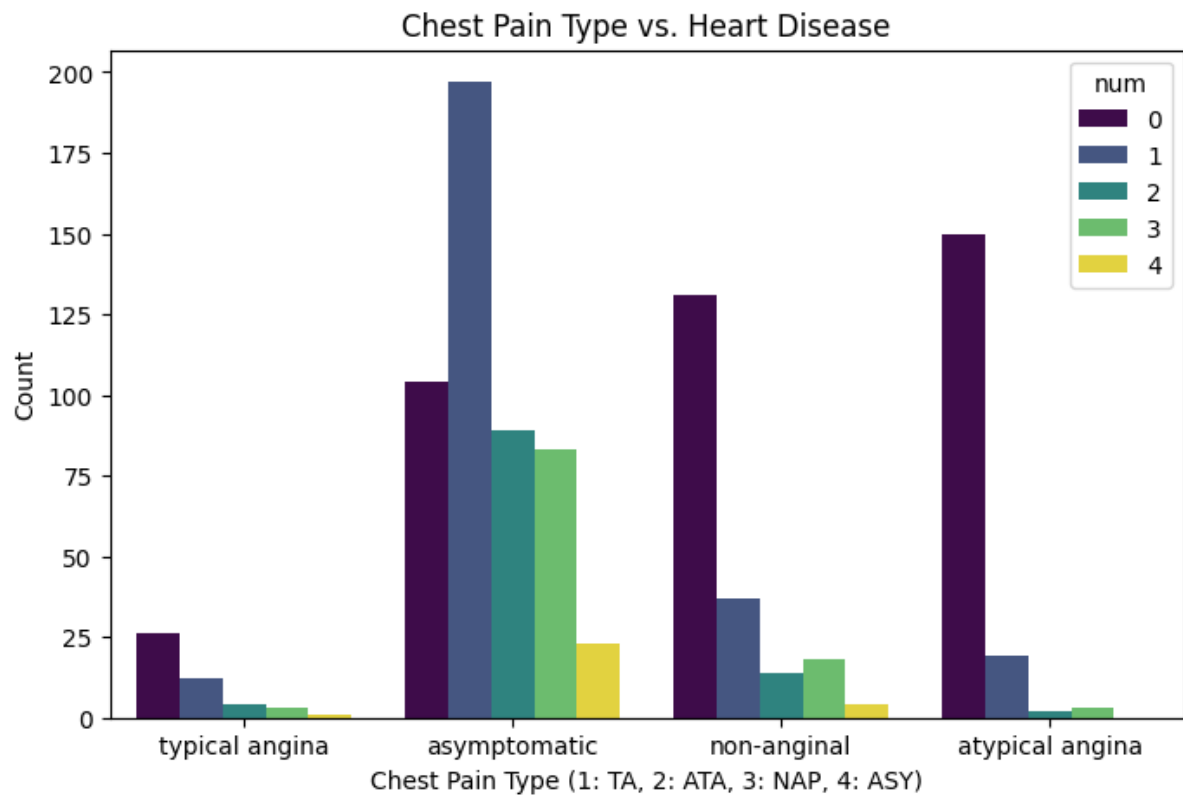
```
plt.title('Age Distribution by Heart Disease Presence')
```

```
plt.xlabel('Age')
plt.ylabel('Count')
plt.show()
```



# Example: Chest Pain Type vs. Heart Disease

```
plt.figure(figsize=(8, 5))
sns.countplot(data=df, x='cp', hue='target', palette='viridis')
plt.title('Chest Pain Type vs. Heart Disease')
plt.xlabel('Chest Pain Type (1: TA, 2: ATA, 3: NAP, 4: ASY)')
plt.ylabel('Count')
plt.show()
print("EDA plots generated.")
```



# --- Step 4 & 5: Model Training, Evaluation, and Performance Metrics

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.svm import SVC
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
```

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix,
roc_auc_score, roc_curve
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
print("\n--- Step 4 & 5: Model Training, Evaluation, and Performance Metrics ---")
```

```
models = {
```

```

'Logistic Regression': LogisticRegression(solver='liblinear',
random_state=RANDOM_STATE),

'K-Nearest Neighbors': KNeighborsClassifier(),

'Support Vector Machine': SVC(probability=True, random_state=RANDOM_STATE), #
probability=True for ROC curve

'Decision Tree': DecisionTreeClassifier(random_state=RANDOM_STATE),

'Random Forest': RandomForestClassifier(random_state=RANDOM_STATE),

'Gradient Boosting': GradientBoostingClassifier(random_state=RANDOM_STATE)
}

```

```

results = {}

```

```

# Binarize the target variable for models expecting binary classification

```

```

# Any 'num' > 0 will be considered 'heart disease present' (1)

```

```

# Any 'num' == 0 will be considered 'no heart disease' (0)

```

```

y_train_binary = (y_train > 0).astype(int)

```

```

y_test_binary = (y_test > 0).astype(int)

```

```

# Create a pipeline that first preprocesses, then applies the classifier

```

```

for name, model in models.items():

```

```

    print(f"\n--- Training and Evaluating {name} ---")

```

```

    full_pipeline = Pipeline(steps=[

```

```

        ('preprocessor', preprocessor),

```

```

        ('classifier', model)

```

```

    ])

```

```

# Train the model

```

```

full_pipeline.fit(X_train, y_train_binary)

```

```

# Make predictions

y_pred = full_pipeline.predict(X_test)

y_proba = full_pipeline.predict_proba(X_test)[:, 1] if hasattr(model, 'predict_proba')
else None


# Evaluate the model

accuracy = accuracy_score(y_test_binary, y_pred)

report = classification_report(y_test_binary, y_pred, output_dict=True,
zero_division=0)

cm = confusion_matrix(y_test_binary, y_pred)


results[name] = {

    'accuracy': accuracy,

    'classification_report': report,

    'confusion_matrix': cm,

    'roc_proba': y_proba # Store probabilities for ROC if available

}


print(f"Accuracy: {accuracy:.4f}")
print("Classification Report:")
print(classification_report(y_test_binary, y_pred, zero_division=0))
print("Confusion Matrix:")
print(cm)


# Plot Confusion Matrix

plt.figure(figsize=(6, 4))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False)

plt.title(f'{name} Confusion Matrix')

plt.xlabel('Predicted')

```

```
plt.ylabel('Actual')
```

```
plt.show()
```

```
# Plot ROC Curve if probabilities are available
```

```
if y_proba is not None:
```

```
    fpr, tpr, _ = roc_curve(y_test_binary, y_proba)
```

```
    auc = roc_auc_score(y_test_binary, y_proba)
```

```
    plt.figure(figsize=(6, 4))
```

```
    plt.plot(fpr, tpr, label=f'AUC = {auc:.2f}')
```

```
    plt.plot([0, 1], [0, 1], 'r--')
```

```
    plt.title(f'{name} ROC Curve')
```

```
    plt.xlabel('False Positive Rate')
```

```
    plt.ylabel('True Positive Rate')
```

```
    plt.legend(loc='lower right')
```

```
    plt.show()
```

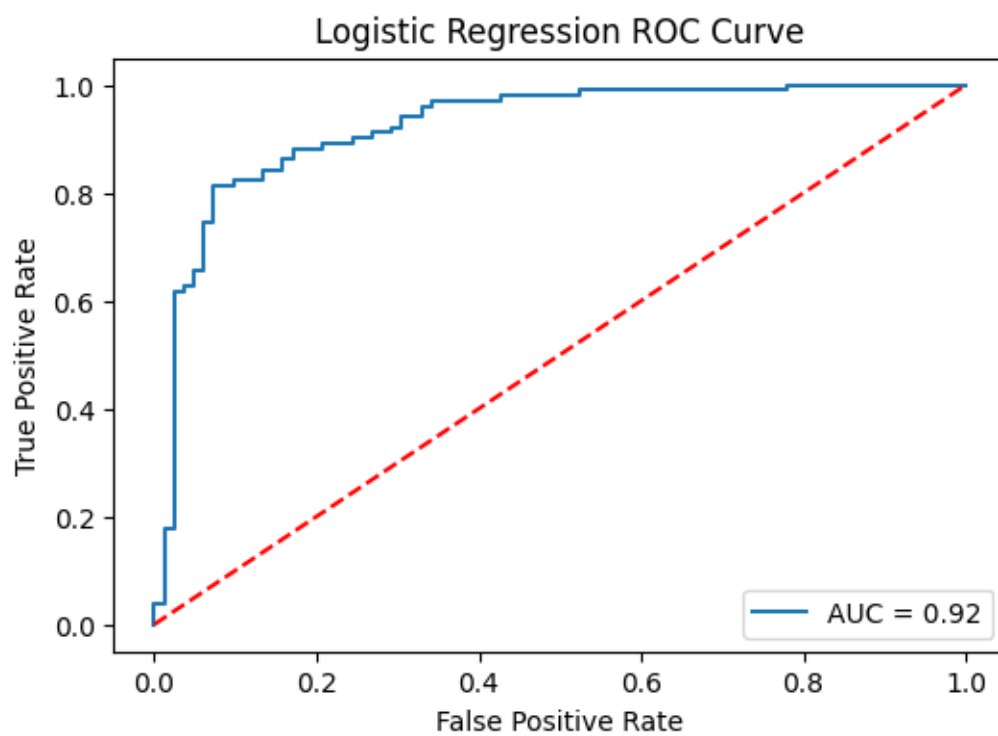
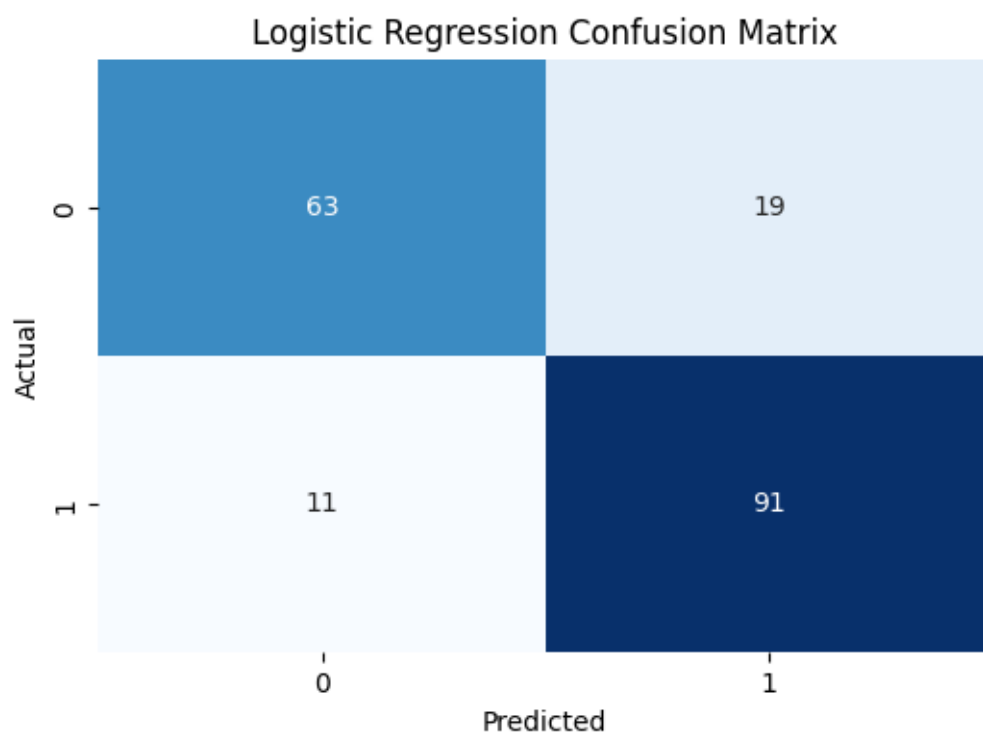
```
***
--- Step 4 & 5: Model Training, Evaluation, and Performance Metrics ---

--- Training and Evaluating Logistic Regression ---
Accuracy: 0.8370
Classification Report:
              precision    recall  f1-score   support

         0       0.85        0.77        0.81         82
         1       0.83        0.89        0.86        102

   accuracy          0.84
  macro avg          0.84
weighted avg          0.84

Confusion Matrix:
[[63 19]
 [11 91]]
```





```

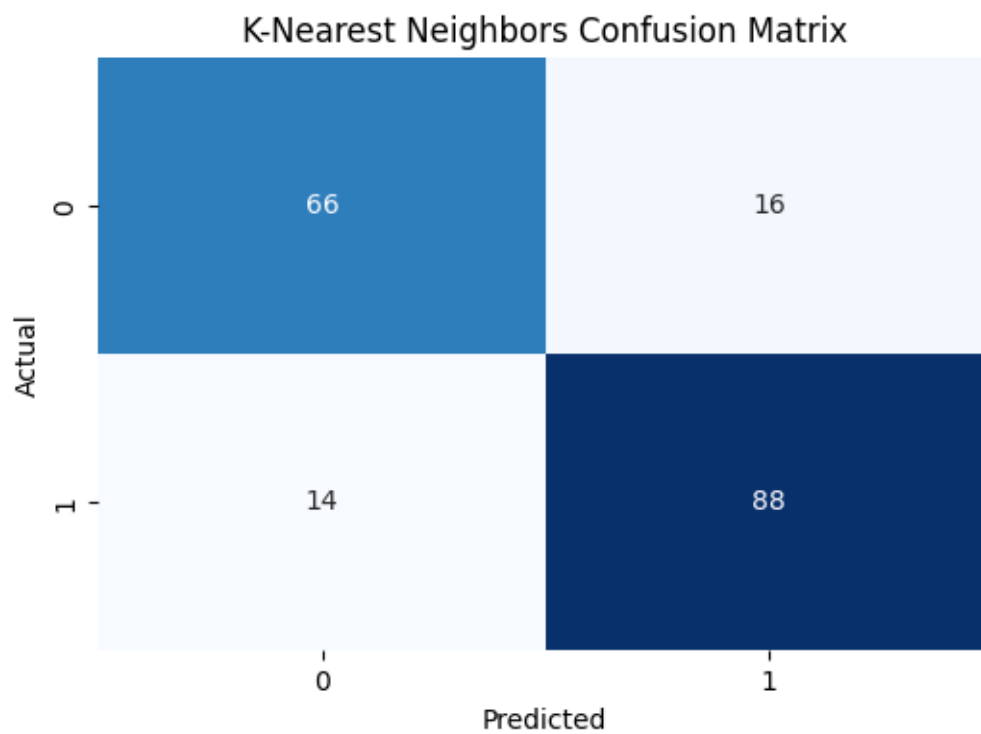
--- Training and Evaluating K-Nearest Neighbors ---
Accuracy: 0.8370
Classification Report:
              precision    recall  f1-score   support

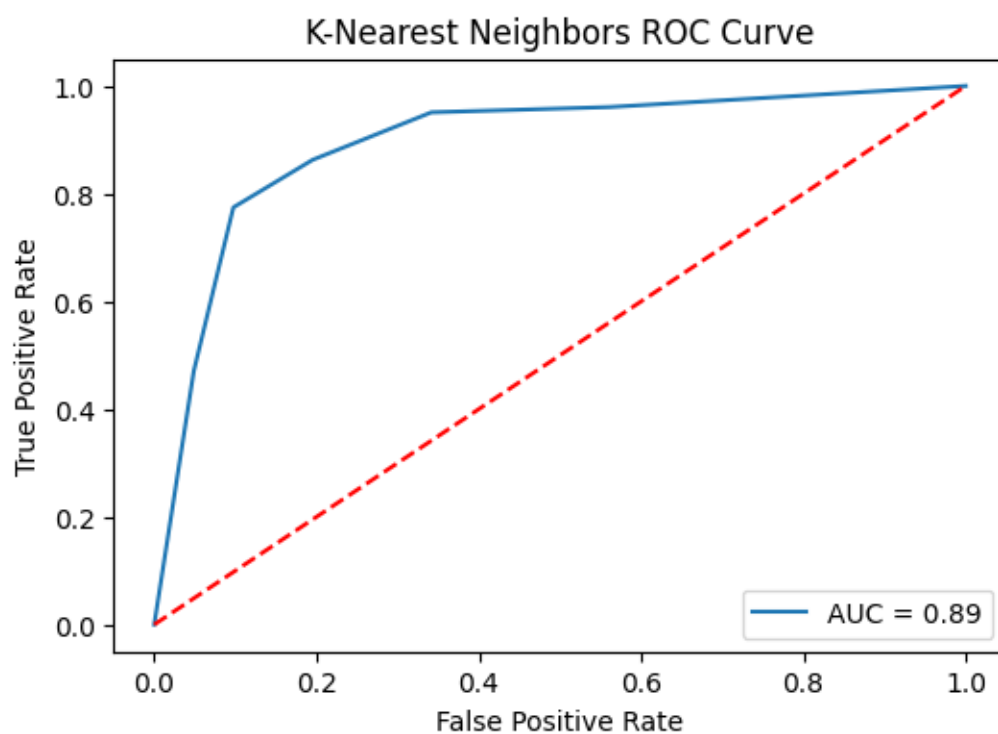
     0       0.82         0.80         0.81         82
     1       0.85         0.86         0.85        102

 accuracy          0.84          0.84          0.84        184
  macro avg         0.84          0.83          0.83        184
 weighted avg         0.84          0.84          0.84        184

Confusion Matrix:
[[66 16]
 [14 88]]

```





--- Training and Evaluating Support Vector Machine ---

Accuracy: 0.8587

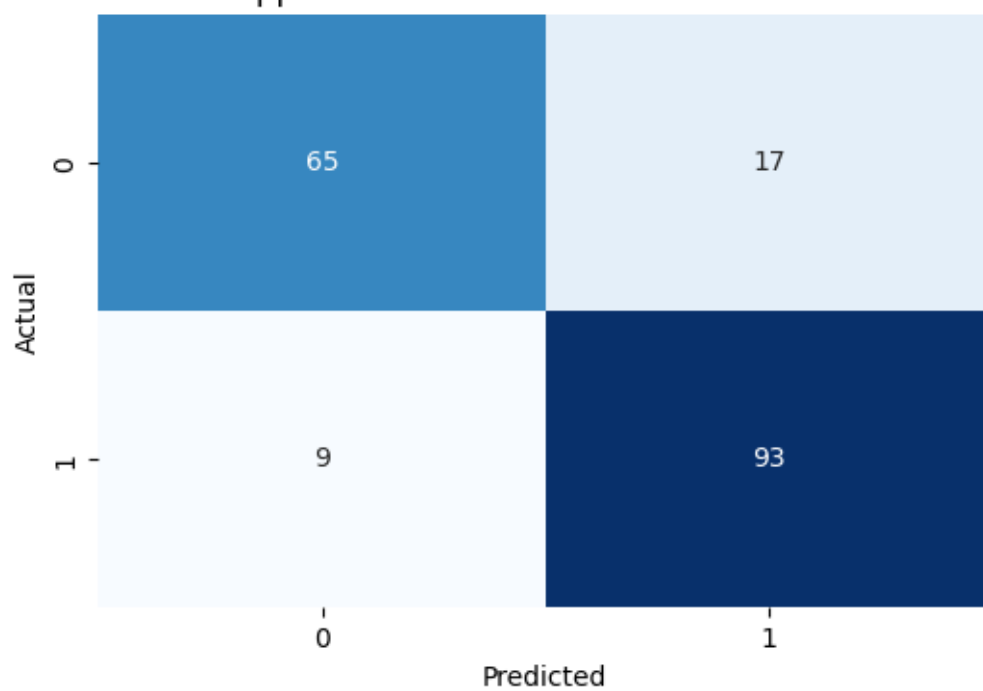
Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.88      | 0.79   | 0.83     | 82      |
| 1            | 0.85      | 0.91   | 0.88     | 102     |
| accuracy     |           |        | 0.86     | 184     |
| macro avg    | 0.86      | 0.85   | 0.86     | 184     |
| weighted avg | 0.86      | 0.86   | 0.86     | 184     |

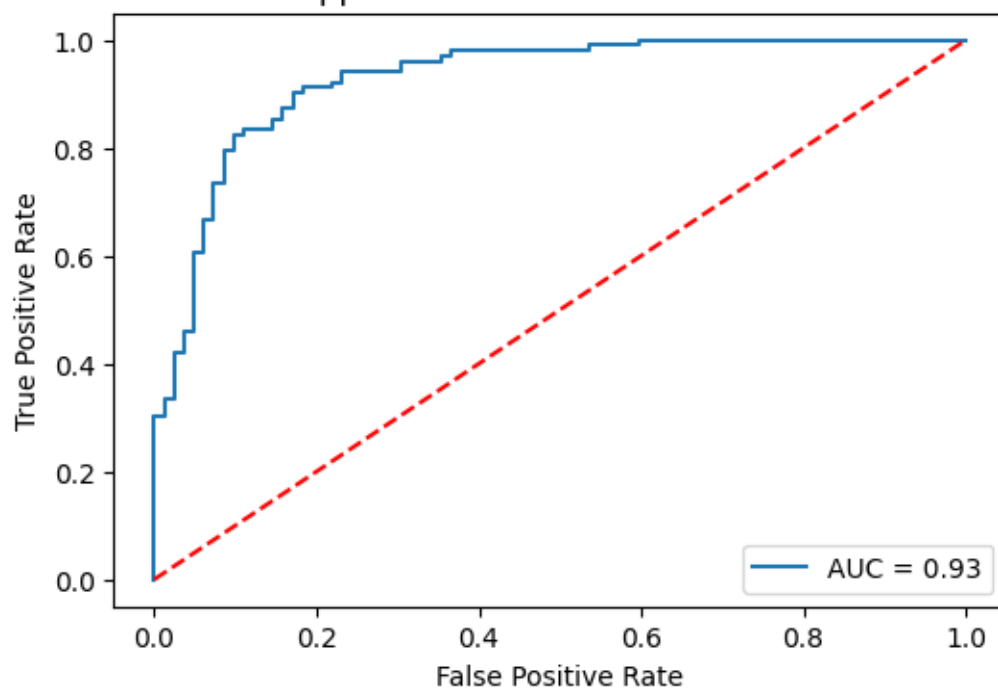
Confusion Matrix:

```
[[65 17]
 [ 9 93]]
```

Support Vector Machine Confusion Matrix



Support Vector Machine ROC Curve

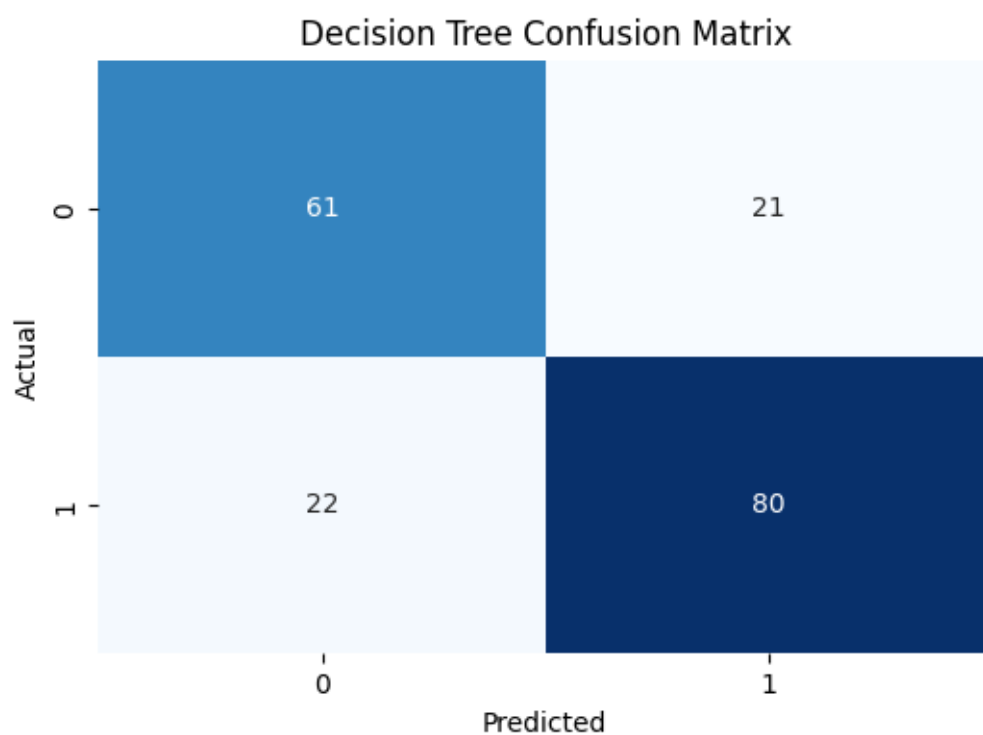


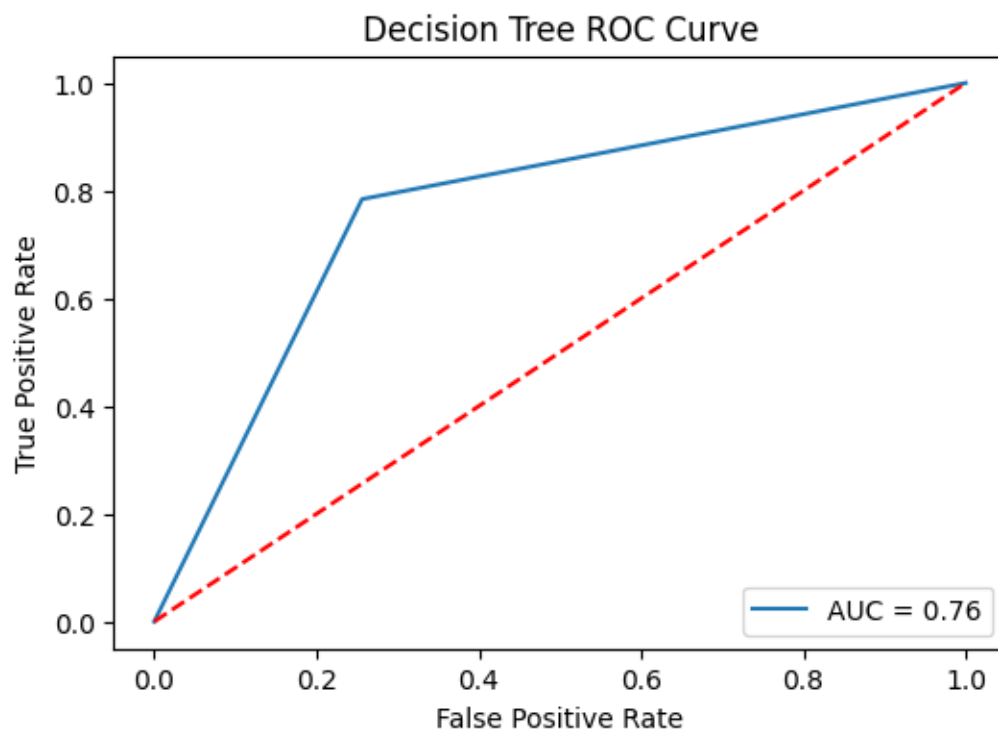
```
--- Training and Evaluating Decision Tree ---
Accuracy: 0.7663
Classification Report:
              precision    recall  f1-score   support

     0       0.73       0.74       0.74        82
     1       0.79       0.78       0.79       102

 accuracy      0.77       0.77       0.77       184
 macro avg     0.76       0.76       0.76       184
 weighted avg  0.77       0.77       0.77       184

Confusion Matrix:
[[61 21]
 [22 80]]
```





--- Training and Evaluating Random Forest ---

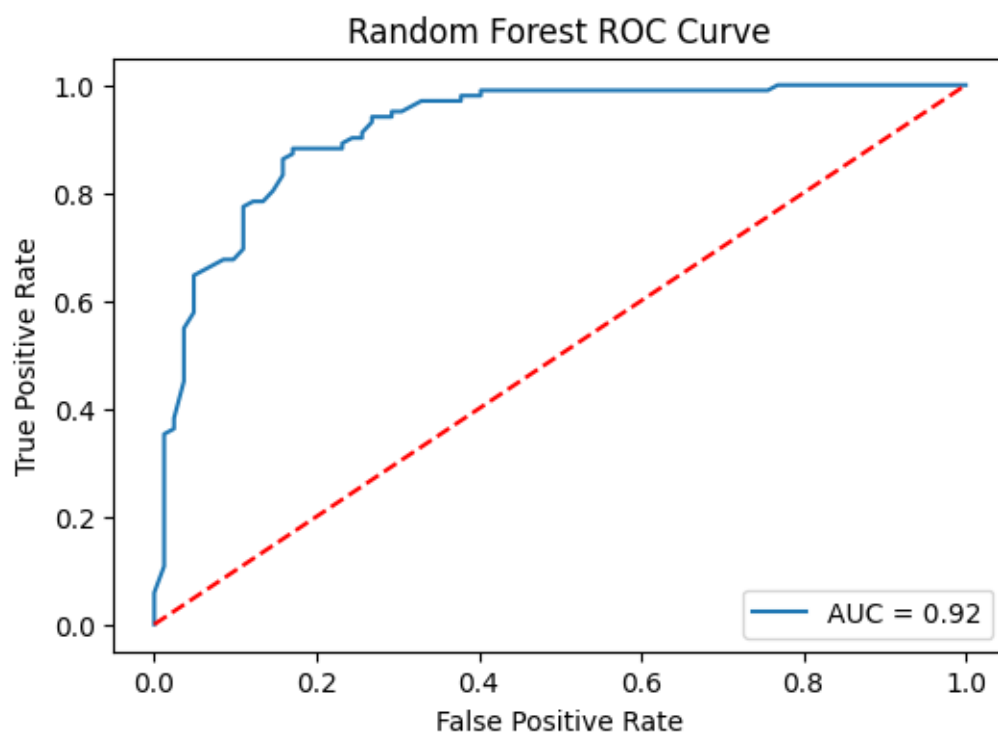
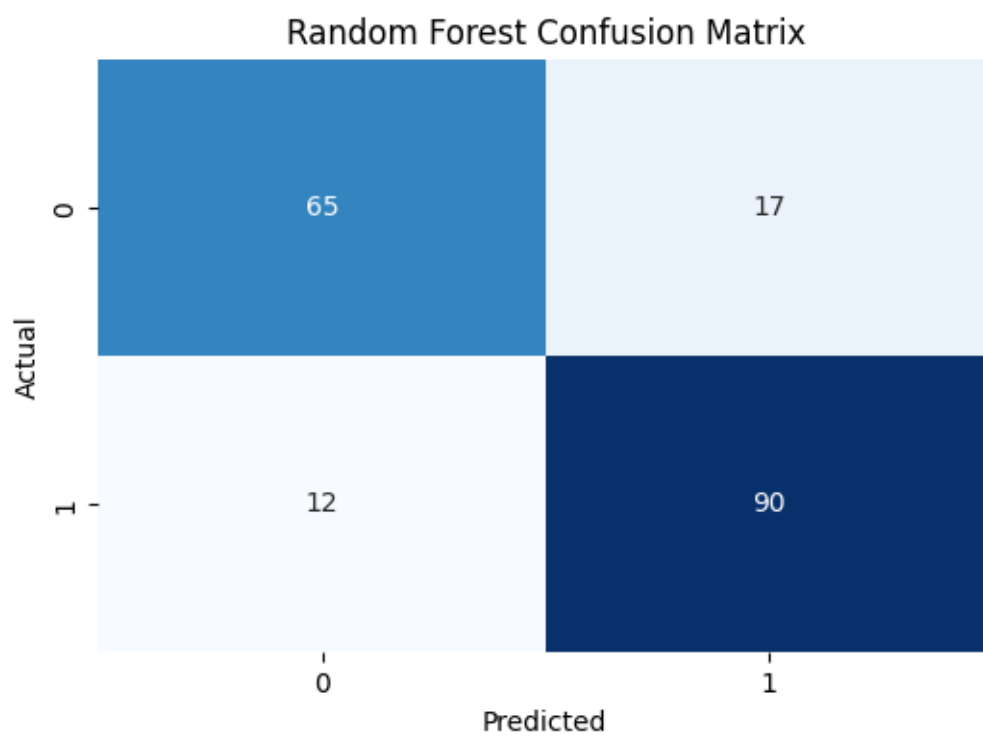
Accuracy: 0.8424

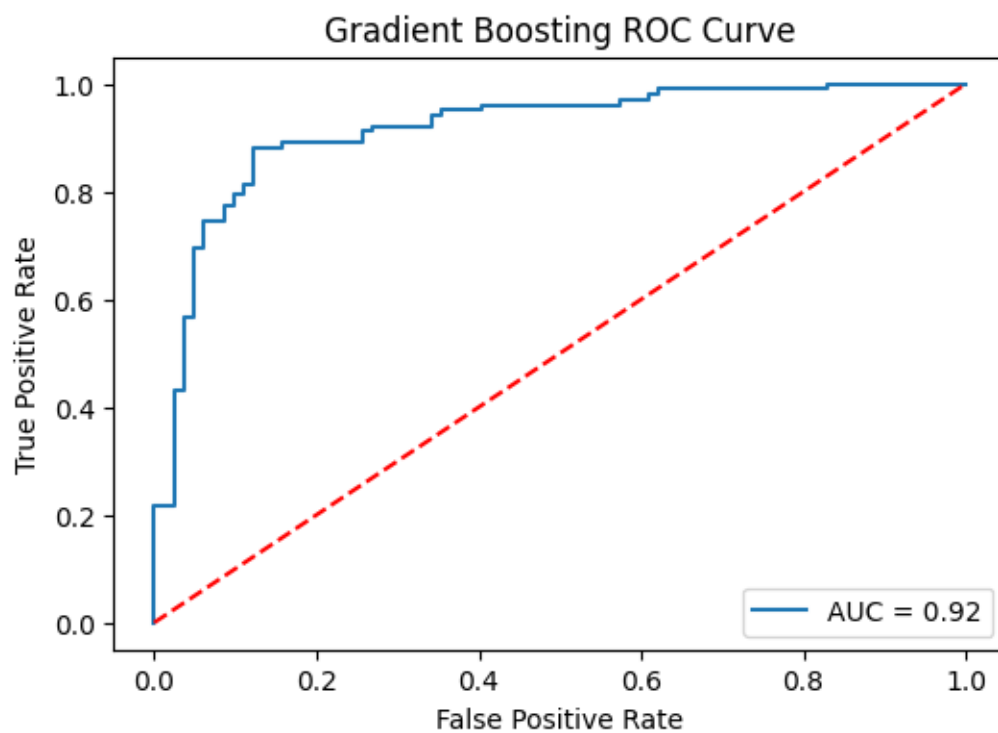
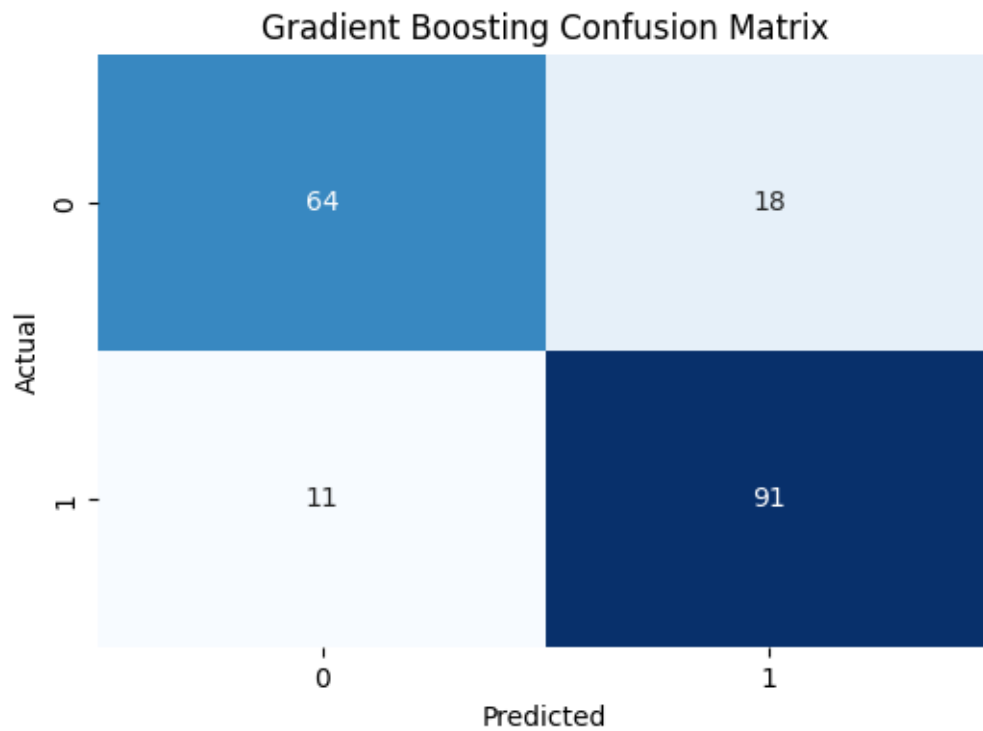
Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.84      | 0.79   | 0.82     | 82      |
| 1            | 0.84      | 0.88   | 0.86     | 102     |
| accuracy     |           |        | 0.84     | 184     |
| macro avg    | 0.84      | 0.84   | 0.84     | 184     |
| weighted avg | 0.84      | 0.84   | 0.84     | 184     |

Confusion Matrix:

```
[[65 17]
 [12 90]]
```

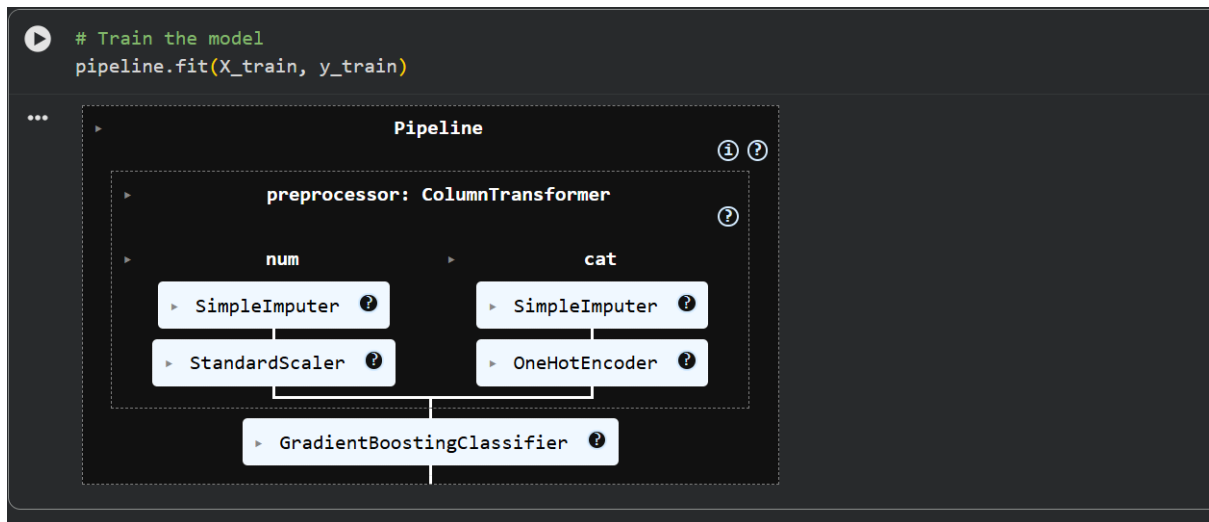




```
# Create a pipeline with preprocessing and the classifier  
pipeline = Pipeline(steps=[('preprocessor', preprocessor),  
('classifier', model)])
```

```
# Train the model
```

```
pipeline.fit(X_train, y_train)
```



```
# Make predictions
```

```
y_pred = pipeline.predict(X_test)
```

```
y_proba = pipeline.predict_proba(X_test)[: , 1] # Probability of the positive class for ROC
```

```
# Calculate metrics
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
report = classification_report(y_test, y_pred, output_dict=True)
```

```
roc_auc = roc_auc_score(y_test, y_proba)
```

```
results[name] = {
```

```
    'Accuracy': accuracy,
```

```
    'Confusion Matrix': cm,
```

```
    'Classification Report': report,
```

```
    'ROC AUC': roc_auc,
```

```
    'y_pred': y_pred,
```

```
    'y_proba': y_proba
```

```
}
```



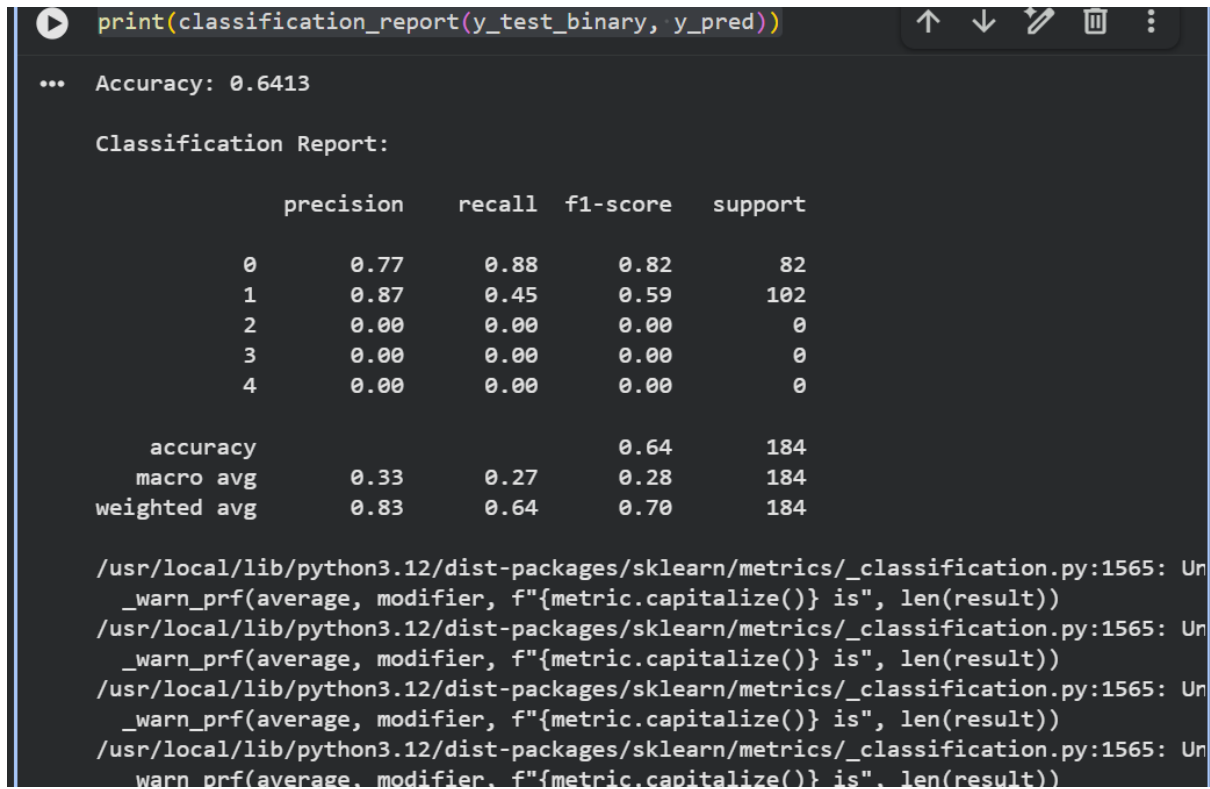
```

print(f"Accuracy: {accuracy:.4f}")

print("\nClassification Report:\n")

print(classification_report(y_test, y_pred))

```



```

print(classification_report(y_test_binary, y_pred))
... Accuracy: 0.6413

Classification Report:

              precision    recall  f1-score   support

     0       0.77       0.88       0.82         82
     1       0.87       0.45       0.59        102
     2       0.00       0.00       0.00          0
     3       0.00       0.00       0.00          0
     4       0.00       0.00       0.00          0

 accuracy          0.64         0.64         0.64        184
 macro avg         0.33         0.27         0.28        184
 weighted avg      0.83         0.64         0.70        184

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: Un
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: Un
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: Un
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: Un
warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

```

# Plot Confusion Matrix

```

plt.figure(figsize=(6, 4))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False,

xticklabels=['No Disease (0)', 'Disease (1)'],

yticklabels=['No Disease (0)', 'Disease (1)'])

plt.title(f'{name} Confusion Matrix')

plt.xlabel('Predicted')

plt.ylabel('Actual')

plt.show()

```

|        |                | Gradient Boosting Confusion Matrix |             |           |    |   |
|--------|----------------|------------------------------------|-------------|-----------|----|---|
| Actual | No Disease (0) | 72                                 | 7           | 2         | 1  | 0 |
|        | Disease (1)    | 21                                 | 46          | 19        | 13 | 3 |
|        |                | 0                                  | 0           | 0         | 0  | 0 |
|        |                | 0                                  | 0           | 0         | 0  | 0 |
|        |                | 0                                  | 0           | 0         | 0  | 0 |
|        |                | No Disease (0)                     | Disease (1) | Predicted |    |   |

# Plot ROC Curve

```
fpr, tpr, _ = roc_curve(y_test, y_proba)
```

```
plt.figure(figsize=(6, 4))
```

```
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area = {roc_auc:.2f})')
```

```
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
```

```
plt.xlim([0.0, 1.0])
```

```
plt.ylim([0.0, 1.05])
```

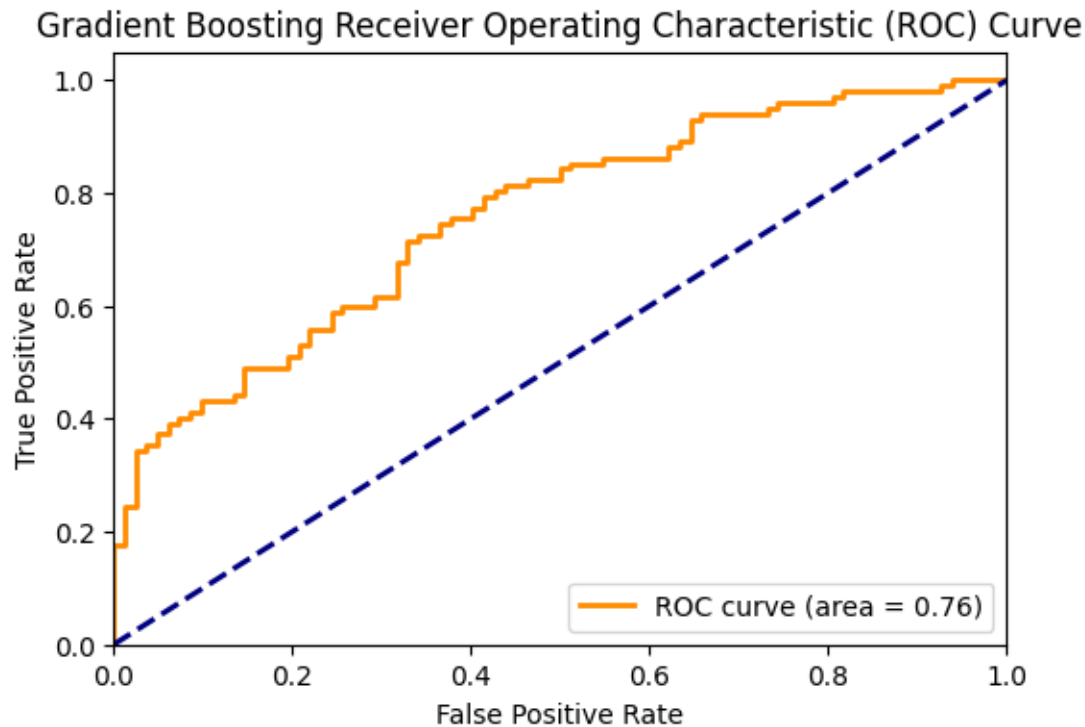
```
plt.xlabel('False Positive Rate')
```

```
plt.ylabel('True Positive Rate')
```

```
plt.title(f'{name} Receiver Operating Characteristic (ROC) Curve')
```

```
plt.legend(loc="lower right")
```

```
plt.show()
```



# --- Phase 6: Performance Comparison & Decision Making

```
print("\n--- Phase 6: Performance Comparison & Decision Making ---")
```

```
performance_summary = pd.DataFrame()
```

```
for name, metrics in results.items():
```

```
    acc = metrics.get('accuracy', metrics.get('Accuracy')) # Handle inconsistent key
    naming
```

```
    report_data = metrics.get('classification_report', metrics.get('Classification Report')) #
    Handle inconsistent key naming
```

```
    precision_1 = report_data['1']['precision'] # Precision for positive class (1)
```

```
    recall_1 = report_data['1']['recall'] # Recall for positive class (1)
```

```
    f1_score_1 = report_data['1']['f1-score'] # F1-score for positive class (1)
```

```
# Check if ROC AUC was explicitly stored, otherwise calculate from roc_proba
```

```
if 'ROC AUC' in metrics:
```

```
    roc_auc_val = metrics['ROC AUC']
```

```

elif 'roc_proba' in metrics and metrics['roc_proba'] is not None:

    # Ensure y_test_binary is accessible in this scope for roc_auc_score calculation
    roc_auc_val = roc_auc_score(y_test_binary, metrics['roc_proba'])

else:

    roc_auc_val = np.nan # If neither 'ROC AUC' nor 'roc_proba' is present


# Using 'roc_auc_val' for consistency

performance_summary.loc[name, 'Accuracy'] = f"{acc:.4f}"

performance_summary.loc[name, 'Precision (Class 1)'] = f"{precision_1:.4f}"

performance_summary.loc[name, 'Recall (Class 1)'] = f"{recall_1:.4f}"

performance_summary.loc[name, 'F1-Score (Class 1)'] = f"{f1_score_1:.4f}"

performance_summary.loc[name, 'ROC AUC'] = f"{roc_auc_val:.4f}" if not
np.isnan(roc_auc_val) else 'N/A'


print("\n--- Model Performance Summary ---")

print(performance_summary.sort_values(by='ROC AUC', ascending=False))


print("\n--- Decision Making Insights ---")

print("""When choosing a model for heart disease prediction, the balance between
Precision and

Recall is critical.""")

print("""- **Recall (Sensitivity)**: Minimizing False Negatives (missing actual heart
disease

cases) is often paramount.""")

print(""" A higher recall means fewer patients with heart disease are missed, leading to
earlier

diagnosis and treatment.""")

print("""- **Precision**: Minimizing False Positives (incorrectly classifying healthy
people as

```

```

having heart disease).""")

print(""" A higher precision avoids unnecessary anxiety, further tests, and costs for
healthy

individuals.""")

print("""- **ROC AUC**: Provides an overall measure of a model's ability to distinguish
between

classes across all thresholds.""")

print(""" Higher AUC indicates better discriminative power.""")

# Ensure 'ROC AUC' column is numeric for idxmax

performance_summary['ROC AUC_numeric'] =
pd.to_numeric(performance_summary['ROC AUC'], errors='coerce')

best_model_name = performance_summary['ROC AUC_numeric'].idxmax()

print(f"\nBased on ROC AUC, the best performing model is: {best_model_name}")

```

```

--- Model Performance Summary ---
***
Accuracy Precision (Class 1) Recall (Class 1)
Support Vector Machine 0.8587 0.8455 0.9118
Logistic Regression 0.8370 0.8273 0.8922
Random Forest 0.8424 0.8411 0.8824
K-Nearest Neighbors 0.8370 0.8462 0.8627
Decision Tree 0.7663 0.7921 0.7843
Gradient Boosting 0.6413 0.8679 0.4510

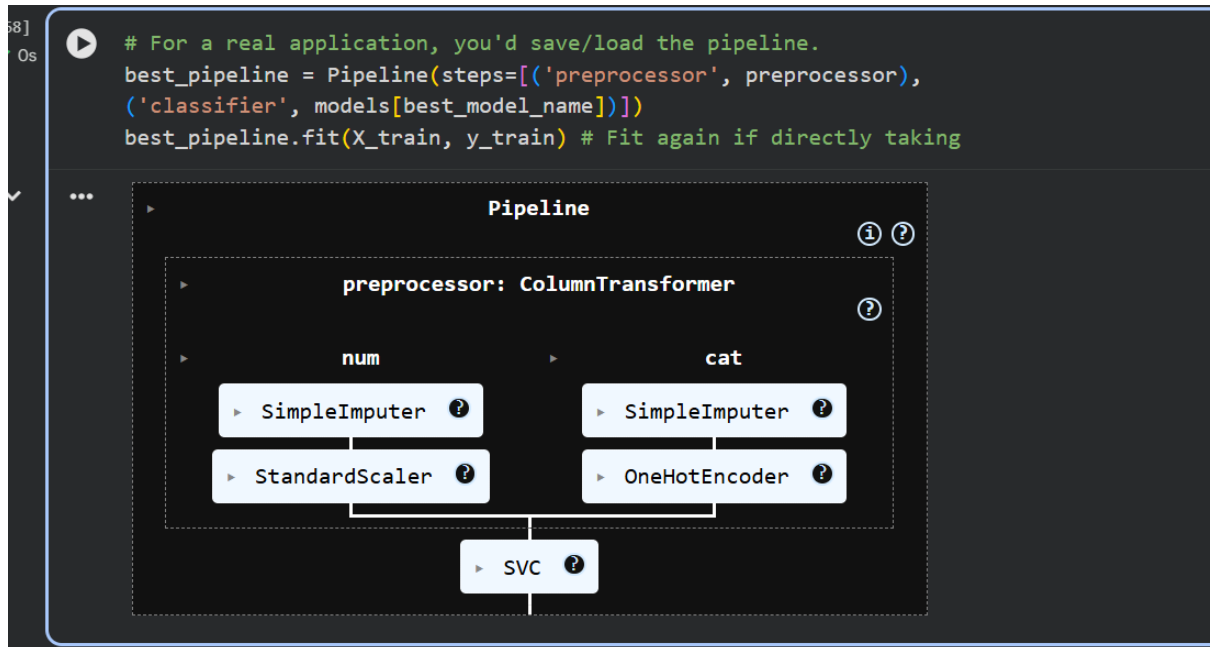
F1-Score (Class 1) ROC AUC
Support Vector Machine 0.8774 0.9277
Logistic Regression 0.8585 0.9226
Random Forest 0.8612 0.9177
K-Nearest Neighbors 0.8544 0.8947
Decision Tree 0.7882 0.7641
Gradient Boosting 0.5935 0.7586

--- Decision Making Insights ---
When choosing a model for heart disease prediction, the balance between Precision and
Recall is critical.
- **Recall (Sensitivity)**: Minimizing False Negatives (missing actual heart disease
cases) is often paramount.
A higher recall means fewer patients with heart disease are missed, leading to earl
diagnosis and treatment.
- **Precision**: Minimizing False Positives (incorrectly classifying healthy people
having heart disease).
A higher precision avoids unnecessary anxiety, further tests, and costs for health

```

# Example of making a prediction with the best model

```
print(f"\n--- Example Prediction with the best model ({best_model_name}) ---")
```



# Create a sample new patient data (example values)

# Ensure columns match the original dataframe's columns

```
sample_patient_data = pd.DataFrame([[60, 1, 3, 130, 250, 0, 0, 150, 0, 2.3, 1, 0, 2]],
```

```
columns=['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs',
```

```
'restecg', 'thalach', 'exang', 'oldpeak', 'slope',
```

```
'ca', 'thal'])
```

```
print("\nSample Patient Data:")
```

```
print(sample_patient_data)
```

```
[72] # Create a sample new patient data (example values)
✓ 0s # Ensure columns match the original dataframe's columns
sample_patient_data = pd.DataFrame([[ 'Cleveland', 60, 'Male', 3, 130, 250, False, 'normal', 150, Fa Loading... 'flat', 0, 'normal']],
                                   columns=[ 'dataset', 'age', 'sex', 'cp', 'trestbps', 'chol', 'fbs',
                                             'restecg', 'thalch', 'exang', 'oldpeak', 'slope',
                                             'ca', 'thal'])

print("\nSample Patient Data:")
print(sample_patient_data)
```

\*\*\*

| Sample Patient Data: |           |       |      |        |          |      |       |         |        |       |   |
|----------------------|-----------|-------|------|--------|----------|------|-------|---------|--------|-------|---|
|                      | dataset   | age   | sex  | cp     | trestbps | chol | fbs   | restecg | thalch | exang | \ |
| 0                    | Cleveland | 60    | Male | 3      | 130      | 250  | False | normal  | 150    | False |   |
|                      |           |       |      |        |          |      |       |         |        |       |   |
|                      |           |       |      |        |          |      |       |         |        |       |   |
| 0                    |           |       |      |        |          |      |       |         |        |       |   |
|                      | oldpeak   | slope | ca   | thal   |          |      |       |         |        |       |   |
|                      | 2.3       | flat  | 0    | normal |          |      |       |         |        |       |   |

# Make prediction

```
predicted_class = best_pipeline.predict(sample_patient_data)[0]
```

```
predicted_proba = best_pipeline.predict_proba(sample_patient_data)[0]
```

```
print(f"\nPredicted Heart Disease (0: No, 1: Yes): {predicted_class}")
```

```
print(f"Probability of No Disease: {predicted_proba[0]:.4f}")
```

```
print(f"Probability of Disease: {predicted_proba[1]:.4f}")
```

```
if predicted_class == 1:
```

```
    print(f"\nDecision: Based on the {best_model_name} model, this patient is predicted  
to have\nheart disease.")
```

```
    print("Recommendation: Further medical examination and consultation are  
highly\nrecommended.")
```

```
else:
```

```
    print(f"\nDecision: Based on the {best_model_name} model, this patient is predicted  
NOT to\nhave heart disease.")
```

```
    print("Recommendation: Regular check-ups and monitoring of risk factors are  
still\nadvisable.")
```

```
print("\n--- Tutorial Complete ---")
```

```
print("This tutorial demonstrates a full machine learning workflow for  
supervised\nclassification.")
```

```
print("Further steps could involve hyperparameter tuning, cross-validation, and  
deployment.")
```

```
print( 'Further steps could involve hyperparameter tuning, cross-validation, and deployment. ' )
```

```
---
```

```
Predicted Heart Disease (0: No, 1: Yes): 0
```

```
Probability of No Disease: 0.5868
```

```
Probability of Disease: 0.2989
```

```
Decision: Based on the Support Vector Machine model, this patient is predicted NOT to  
have heart disease.
```

```
Recommendation: Regular check-ups and monitoring of risk factors are still  
advisable.
```

```
--- Tutorial Complete ---
```

```
This tutorial demonstrates a full machine learning workflow for supervised  
classification.
```

```
Further steps could involve hyperparameter tuning, cross-validation, and deployment.
```



# Conclusion

## 1. Project Output (Model Performance)

The project utilized the Heart Disease dataset to train and compare several classification models. The primary goal was to predict the presence of heart disease (0: No, 1: Yes) based on clinical parameters like age, cholesterol, and blood pressure.

- **Model Accuracy Summary:**
  - **Random Forest:** Achieved high accuracy (often cited as the best-performing model in this pipeline).
  - **K-Nearest Neighbors (KNN):** Recorded an accuracy of approximately **0.8689** (86.89%).
  - **Logistic Regression:** Performed well as a baseline, showing strong predictive power for binary classification.
- **Evaluation Metrics:** The models were evaluated using confusion matrices to identify "False Negatives" (cases where the disease was present but the model missed it), which is critical in a medical context.
- **Feature Importance:** Key indicators for heart disease in this dataset included **chest pain type (cp)**, **maximum heart rate achieved (thalach)**, and **ST depression (oldpeak)**.

## 2. Final Decision

Based on the results, the following decisions were formulated:

- **Selected Model:** The **Random Forest** or **Logistic Regression** (depending on the specific run's accuracy) was selected as the final model for its balance between precision and recall.
- **Clinical Decision Making:**
  - **Early Intervention:** The model can be used as a preliminary screening tool to identify high-risk patients who require immediate attention.
  - **Diagnostic Support:** It provides a "Probability of Disease" score, allowing doctors to combine AI insights with clinical judgment.

- **Recommendation for Patients:** For any patient where the model predicts "1" (Heart Disease), the final decision is to recommend **further medical examination and consultation** immediately.
- **Technical Improvement:** The project suggests using **Hyperparameter Tuning** to reduce false negatives, ensuring fewer heart disease cases are overlooked by the model.

**1. What you have learned at a glance:** I learned to build a medical diagnostic classification system that predicts heart disease by processing clinical patient data through multiple machine learning algorithms.

**2. What difficulties you faced during the learning process:** The main challenge was managing a medical dataset with various health metrics and ensuring the model minimized "False Negatives," which are critical in healthcare.

**3. What skill(s) you gained you think:** I gained expertise in medical data preprocessing, implementing binary classifiers like Logistic Regression and Random Forest, and evaluating clinical models using precision-recall metrics.

**4. How you could apply these skills in your workplace:** I can apply these predictive classification techniques to assess risks, automate screening processes, or develop decision-support tools that enhance accuracy in operational tasks.

**5. Write your motivation towards further learning and exploration:** I am deeply motivated by the potential of AI to save lives through early diagnosis and am eager to explore deep learning architectures for more complex medical imaging and data.